ssGSEA3.0: from permutation testing to gene-level bootstrap resampling

Contents

Overview . . . 1
Why bootstrap instead of permutation? . . . 1
Example 1: ssGSEA on tumor NMF factors . . . 2
Example 2: the same tumor NMF factors, scored against REACTOME . . . 5
Example 3: PTM-SEA on a PI3K-inhibitor perturbation dataset . . . 9
Session info . . . 12

Overview

`ssGSEA3.0` runs single-sample Gene Set Enrichment Analysis (ssGSEA) on gene-centric expression data, and PTM-Signature Enrichment Analysis (PTM-SEA) on site-centric phosphoproteomics data scored against the directional PTM signatures database (PTMsigDB). Significance is assessed via a gene-level bootstrap, replacing the gene-label permutation testing used by the original `ssGSEA2.0`.

This vignette:

1. explains why that replacement happened,
2. walks through two ordinary ssGSEA analyses on the same tumor NMF factors – against the curated HALLMARK collection, and against the much larger and more functionally diverse REACTOME pathway collection – and
3. walks through a directional PTM-SEA analysis (a phosphoproteomics perturbation dataset vs. PTMsigDB), comparing the result to the original `ssGSEA2.0`.

```
library(ssGSEA3)
```

Why bootstrap instead of permutation?

A known problem with weighted scoring under permutation

`ssGSEA2.0`'s significance testing permuted gene labels to build a null distribution for each gene set's enrichment score. This works cleanly when every gene contributes equally to the score (`weight = 0`), but is known to produce badly miscalibrated p-values whenever the score is weighted by actual expression magnitude (`weight > 0`) – documented as GitHub issue #45 against the original `ssGSEA2.0` repository. Testing purely random, biologically meaningless gene sets at `weight = 0.75` gave a mean p-value of about 0.001 (i.e. almost always called "significant") instead of the ~0.5 expected under a true null.

An informal fix – stop reordering the correlation vector along with the permuted gene ranks – corrected that calibration problem, but at a real cost: on a real PI3K-perturbation dataset, the same fix reduced detection of expected cell-cycle pathways at `weight = 0.75` from 6/14 down to 0/14 at 6 hours, and from 10/14 down

to 3/14 at 24 hours. Fixing the false-positive problem this way also suppressed real signal – a fundamental validity-vs-power tension that a purely permutation-based approach could not resolve.

`ssGSEA3.0` circumvents the need for this tradeoff by using bootstrap sampling. In a traditional permutation test, random permutations of the data are used to derive the null distribution, and the p-value is calculated based on how extreme the observed value is compared to the null distribution. Here, we use bootstrap sampling to determine the standard deviation of the observed value, and the p-value is determined akin to a z-test on the observed value (using the bootstrap-derived standard deviation). The effective null distribution is the mean shifted bootstrap values. With this approach, all gene-gene relationships and dependencies are maintained, with the bootstrap providing a perturbation of the enrichment score, thereby minimizing bias and controlling false positives.

Directional (PTM-SEA) scoring gets an extra benefit

PTM-SEA's directional gene sets (with `;u`/`;d`-labeled "up" and "down" members) benefit from bootstrap resampling in a second, independent way. Permutation always preserves the exact up/down membership counts of a gene set, only ever moving where they fall in the ranking. Bootstrap resampling with replacement lets some members drop out (and others duplicate) in any given replicate, so the up- and down-regulated counts genuinely vary from replicate to replicate – a richer, two-sided source of resampling variability with no permutation-based analogue.

A controlled simulation with known ground truth (embedding a real, moderate, correctly-oriented effect) confirmed this: both methods are well-calibrated under a true null, but bootstrap shows a real, statistically robust power advantage specifically for directional scoring (McNemar p = 0.0001, odds ratio 1.85) that does not replicate for non-directional scoring at the same sample size. The two worked examples below show both regimes on real data.

Example 1: ssGSEA on tumor NMF factors

The bundled example is a PANOPLY LUAD (lung adenocarcinoma) NMF factorization (4 factors, CPTAC/ICPC v6.1) scored against the MSigDB HALLMARK collection – an ordinary (non-directional) ssGSEA analysis.

```
res <- ssGSEA3(
  input.ds = "example/LUAD_matrix_W_combined_signed_gc.gct",
  output.prefix = "luad",
  gene.set.databases = "db/msigdb/h.all.v7.0.symbols.gmt",
  sample.norm.type = "rank", weight = 1, correl.type = "z.score",
  statistic = "area.under.RES", min.overlap = 10, nperm = 1000, seed = 42
)
```

```
## the call above takes ~1 min at the default nperm=1000 -- loading its
## precomputed output here keeps this vignette's own build fast, and keeps
## the displayed results consistent with the full-nperm comparison below
## (rather than a faster-but-different reduced-nperm run)
res <- readRDS(system.file("extdata", "luad_ssgsea3_result.rds", package = "ssGSEA3"))
res
#> <ssgsea_result> 50 gene sets x 4 samples
#>   score type: NES
```

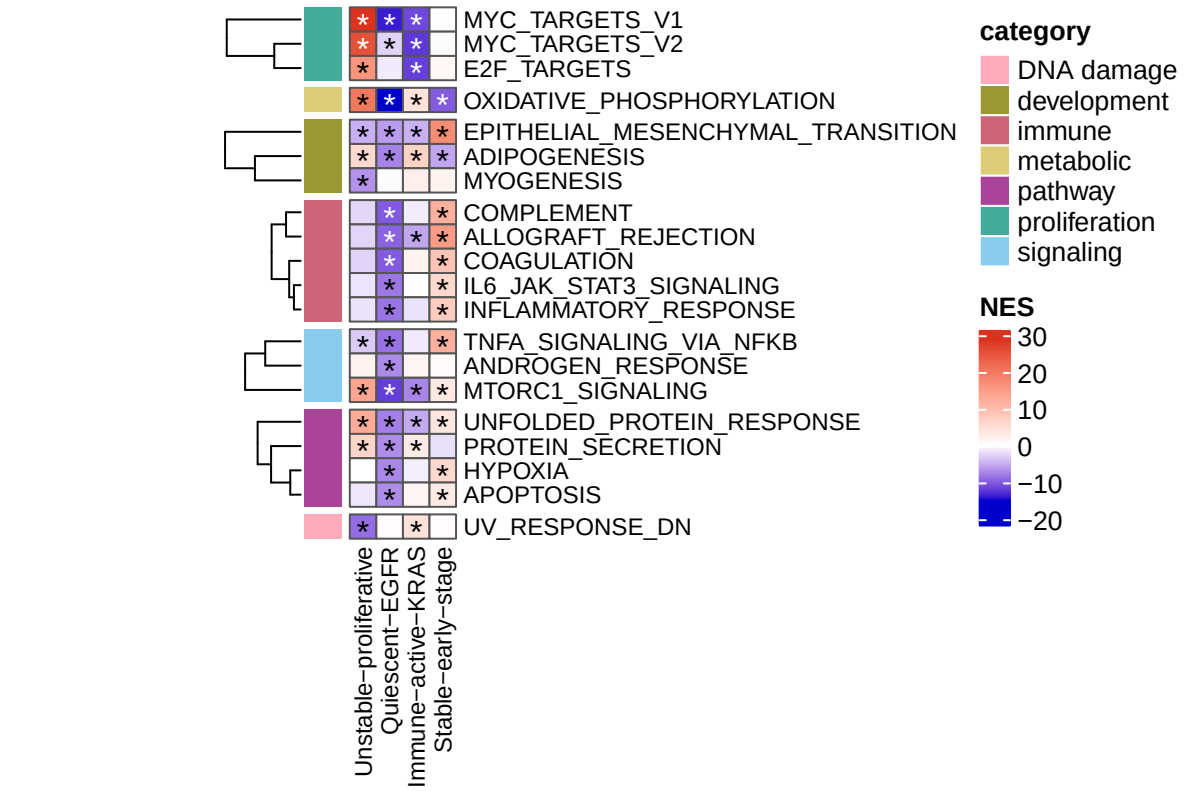
The four factors are labeled by their NMF cluster number; renaming them to their published biological identities makes the heatmap self-explanatory:

```
cluster.names <- c(
  "1" = "Unstable-proliferative", "2" = "Quiescent-EGFR",
  "3" = "Immune-active-KRAS", "4" = "Stable-early-stage"
```

```
)

rename_cols <- function(x, map) {
  colnames(x) <- unname(map[colnames(x)])
  x
}
res$scores <- rename_cols(res$scores, cluster.names)
res$pvalues <- rename_cols(res$pvalues, cluster.names)
res$fdr <- rename_cols(res$fdr, cluster.names)
res$sample.names <- unname(cluster.names[res$sample.names])
```

```
ht <- plot_ssgsea_heatmap(res, top_n = 10, fdr.max = 0.05,
                          title = "LUAD PANOPLY -- HALLMARK (ssGSEA3.0)")
ComplexHeatmap::draw(ht)
```



The proliferation program (`MYC_TARGETS`, `E2F_TARGETS`, `MTORC1_SIGNALING`, `OXIDATIVE_PHOSPHORYLATION`, `GLYCOLYSIS`) is strongly and specifically enriched in the "Unstable-proliferative" factor, matching its published biological identity – a useful sanity check that the scoring and reporting pipeline agree with the underlying biology.

Comparison against the original ssGSEA2.0

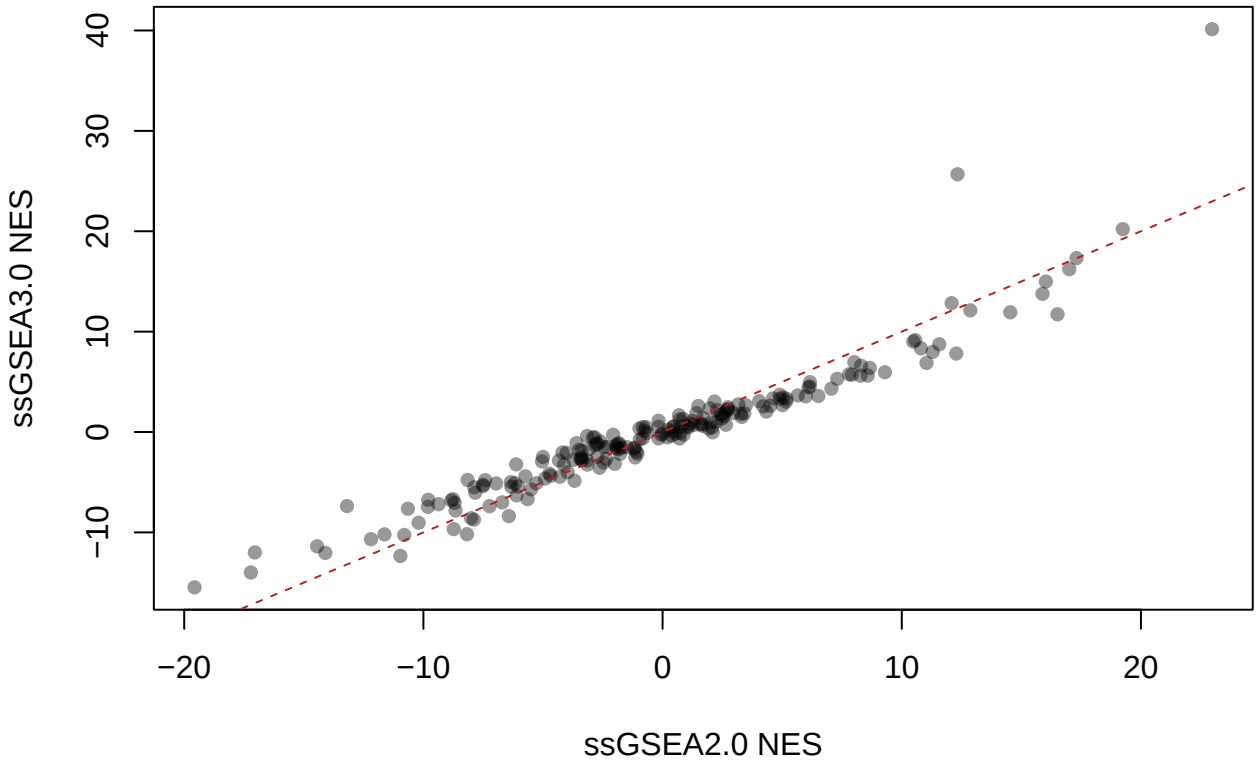
This dataset is scored at `weight = 1` – exactly the regime affected by the weight-dependent calibration issue described above, so it demonstrates the other side of the bootstrap's benefit: rather than finding more real signal (as in the directional PTM-SEA example below), it trims down some of the original ssGSEA2.0's likely-inflated calls while retaining the large majority of them. As above, these are precomputed results (`nperm = 1000`, run outside this vignette):

```r
cmp.luad <- readRDS(system.file(
  "extdata", "luad_ssgsea2_vs_ssgsea3_comparison.rds", package = "ssGSEA3"
))
nes2.luad <- as.vector(cmp.luad$ssgsea2.nes)
nes3.luad <- as.vector(cmp.luad$ssgsea3.nes)

cat("Gene sets compared:", length(cmp.luad$gene.set.names), "\n")
#> Gene sets compared: 50
cat("NES Pearson r:    ", round(cor(nes2.luad, nes3.luad), 3), "\n")
#> NES Pearson r:     0.945
cat("NES Spearman rho: ", round(cor(nes2.luad, nes3.luad, method = "spearman"), 3), "\n")
#> NES Spearman rho:  0.98

## a gene set counts as "significant" if FDR<0.05 in >=1 of the 4 samples --
## NOT the same as counting gene-set x sample cells, which would double-count
## any gene set significant in multiple samples
sig2.luad <- apply(cmp.luad$ssgsea2.fdr < 0.05, 1, any)
sig3.luad <- apply(cmp.luad$ssgsea3.fdr < 0.05, 1, any)
cat("\nGene sets significant (FDR < 0.05 in >=1 sample), of",
 ↪  length(cmp.luad$gene.set.names), "total:\n")
#>
#> Gene sets significant (FDR < 0.05 in >=1 sample), of 50 total:
cat("  ssGSEA2.0:", sum(sig2.luad), " ssGSEA3.0:", sum(sig3.luad),
    " both:", sum(sig2.luad & sig3.luad), " ssGSEA2.0-only:", sum(sig2.luad &
     ↪  !sig3.luad),
    " ssGSEA3.0-only:", sum(!sig2.luad & sig3.luad), "\n")
#>   ssGSEA2.0: 48  ssGSEA3.0: 41  both: 41  ssGSEA2.0-only: 7  ssGSEA3.0-only: 0
```

```r
plot(nes2.luad, nes3.luad,
     xlab = "ssGSEA2.0 NES", ylab = "ssGSEA3.0 NES", pch = 16, col = "#00000066",
     main = "LUAD PANOPLY, HALLMARK: NES agreement")
abline(0, 1, col = "firebrick", lty = 2)
```



`ssGSEA3.0` agrees with `ssGSEA2.0` on the overwhelming majority of significant calls (41 shared), while dropping 7 calls unique to `ssGSEA2.0` and adding almost none of its own – consistent with correcting a false-positive-prone regime rather than disagreeing with it.

Example 2: the same tumor NMF factors, scored against REACTOME

HALLMARK's 50 gene sets are curated to be similarly sized and mechanistically narrow (one canonical process each). REACTOME is far larger and more functionally diverse – gene sets range from a handful of genes up to several hundred, spanning everything from individual metabolic reactions to broad pathway categories – which makes it a useful second, harder test of the same LUAD data: does `ssGSEA3.0` agree with `ssGSEA2.0` as well on a size/function-heterogeneous database as it does on a curated, uniform one?

```
res.reactome <- ssGSEA3(
  input.ds = "example/LUAD_matrix_W_combined_signed_gc.gct",
  output.prefix = "luad_reactome",
  gene.set.databases = "db/msigdb/reactome_subset.gmt",
  sample.norm.type = "rank", weight = 1, correl.type = "z.score",
  statistic = "area.under.RES", min.overlap = 10, nperm = 1000, seed = 42
)
```

```
## as above, loading the precomputed output instead of re-running the
## ~18 min scoring call live -- restricted to the 1,167 gene sets common to
## both engines being compared below
res.reactome <- readRDS(system.file(
  "extdata", "luad_reactome_ssgsea3_result.rds", package = "ssGSEA3"
))
res.reactome
```

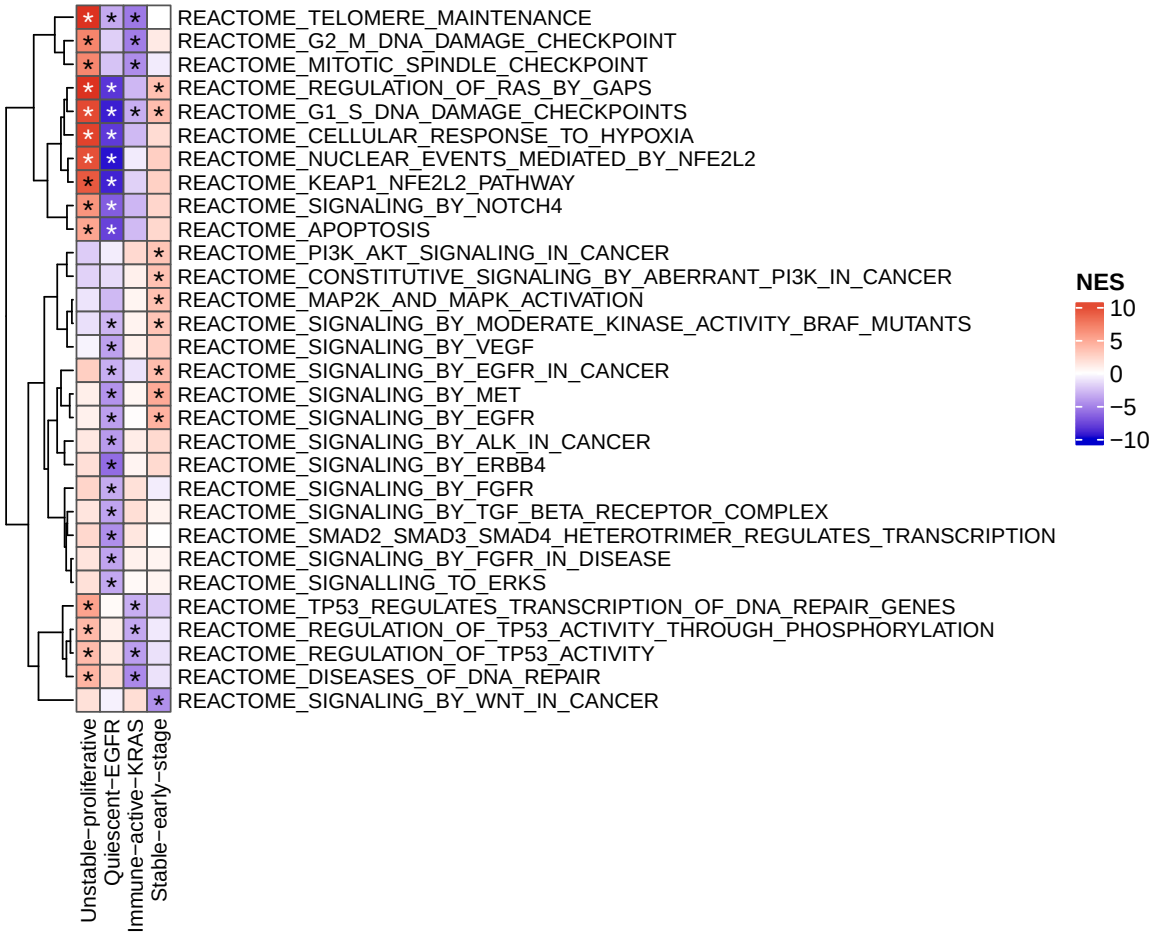
```
#> <ssgsea_result> 1167 gene sets x 4 samples
#>   score type: NES
```

```
res.reactome$scores <- rename_cols(res.reactome$scores, cluster.names)
res.reactome$pvalues <- rename_cols(res.reactome$pvalues, cluster.names)
res.reactome$fdr <- rename_cols(res.reactome$fdr, cluster.names)
res.reactome$sample.names <- unname(cluster.names[res.reactome$sample.names])
```

Auto-selecting top terms by `|NES|` on a size-heterogeneous database like REACTOME tends to surface a handful of very large "umbrella" gene sets (e.g. `METABOLISM_OF_RNA`, `CELL_CYCLE`) rather than the mechanistically specific pathways a reader would actually want to see. Instead, this heatmap uses a curated list of 30 lung adenocarcinoma driver pathways spanning EGFR/ERBB, RAS/RAF/MAPK, ALK, PI3K/AKT, WNT, NOTCH, TGF-beta, TP53, DNA damage/repair, cell-cycle checkpoints, apoptosis, KEAP1-NRF2, telomere maintenance, FGFR, MET, and hypoxia/VEGF signaling:

```
luad.driver.pathways <- c(
  "REACTOME_SIGNALING_BY_EGFR", "REACTOME_SIGNALING_BY_EGFR_IN_CANCER",
  "REACTOME_SIGNALING_BY_ERBB4", "REACTOME_SIGNALING_BY_ALK_IN_CANCER",
  "REACTOME_REGULATION_OF_RAS_BY_GAPS", "REACTOME_SIGNALLING_TO_ERKS",
  "REACTOME_MAP2K_AND_MAPK_ACTIVATION",
  "REACTOME_SIGNALING_BY_MODERATE_KINASE_ACTIVITY_BRAF_MUTANTS",
  "REACTOME_PI3K_AKT_SIGNALING_IN_CANCER",
  "REACTOME_CONSTITUTIVE_SIGNALING_BY_ABERRANT_PI3K_IN_CANCER",
  "REACTOME_SIGNALING_BY_WNT_IN_CANCER", "REACTOME_SIGNALING_BY_NOTCH4",
  "REACTOME_SIGNALING_BY_TGF_BETA_RECEPTOR_COMPLEX",
  "REACTOME_SMAD2_SMAD3_SMAD4_HETEROTRIMER_REGULATES_TRANSCRIPTION",
  "REACTOME_REGULATION_OF_TP53_ACTIVITY",
  "REACTOME_REGULATION_OF_TP53_ACTIVITY_THROUGH_PHOSPHORYLATION",
  "REACTOME_TP53_REGULATES_TRANSCRIPTION_OF_DNA_REPAIR_GENES",
  "REACTOME_G1_S_DNA_DAMAGE_CHECKPOINTS", "REACTOME_G2_M_DNA_DAMAGE_CHECKPOINT",
  "REACTOME_MITOTIC_SPINDLE_CHECKPOINT", "REACTOME_APOPTOSIS",
  "REACTOME_KEAP1_NFE2L2_PATHWAY", "REACTOME_NUCLEAR_EVENTS_MEDIATED_BY_NFE2L2",
  "REACTOME_TELOMERE_MAINTENANCE", "REACTOME_SIGNALING_BY_FGFR",
  "REACTOME_SIGNALING_BY_FGFR_IN_DISEASE", "REACTOME_CELLULAR_RESPONSE_TO_HYPOXIA",
  "REACTOME_SIGNALING_BY_VEGF", "REACTOME_SIGNALING_BY_MET",
  "REACTOME_DISEASES_OF_DNA_REPAIR"
)
ht.reactome <- plot_ssgsea_heatmap(res.reactome, terms = luad.driver.pathways,
                                   fdr.max = 0.05,
                                   title = "LUAD PANOPLY -- REACTOME driver pathways
                                    ↪  (ssGSEA3.0)")
ComplexHeatmap::draw(ht.reactome)
```

−− REACTOME driver pathways (ssGSEA3.0)



All 30 of these are significant (FDR<0.05) in at least one of the four NMF factors. Cell-cycle and DNA-damage-checkpoint pathways (`G1_S_DNA_DAMAGE_CHECKPOINTS`, `TP53_REGULATES_TRANSCRIPTION_OF_DNA_REPAIR_GENES`, `MITOTIC_SPINDLE_CHECKPOINT`) are strongly and specifically enriched in the “Unstable-proliferative” factor, matching its published biological identity – the same kind of biology-matches-label sanity check as the HALLMARK example above, now visible on a much larger and more heterogeneous database.

Comparison against the original ssGSEA2.0

As in the HALLMARK example, this dataset is scored at `weight = 1`. REACTOME’s size and functional diversity make the calibration/power tradeoff described above far more visible than it is on HALLMARK’s uniform, curated 50 gene sets:

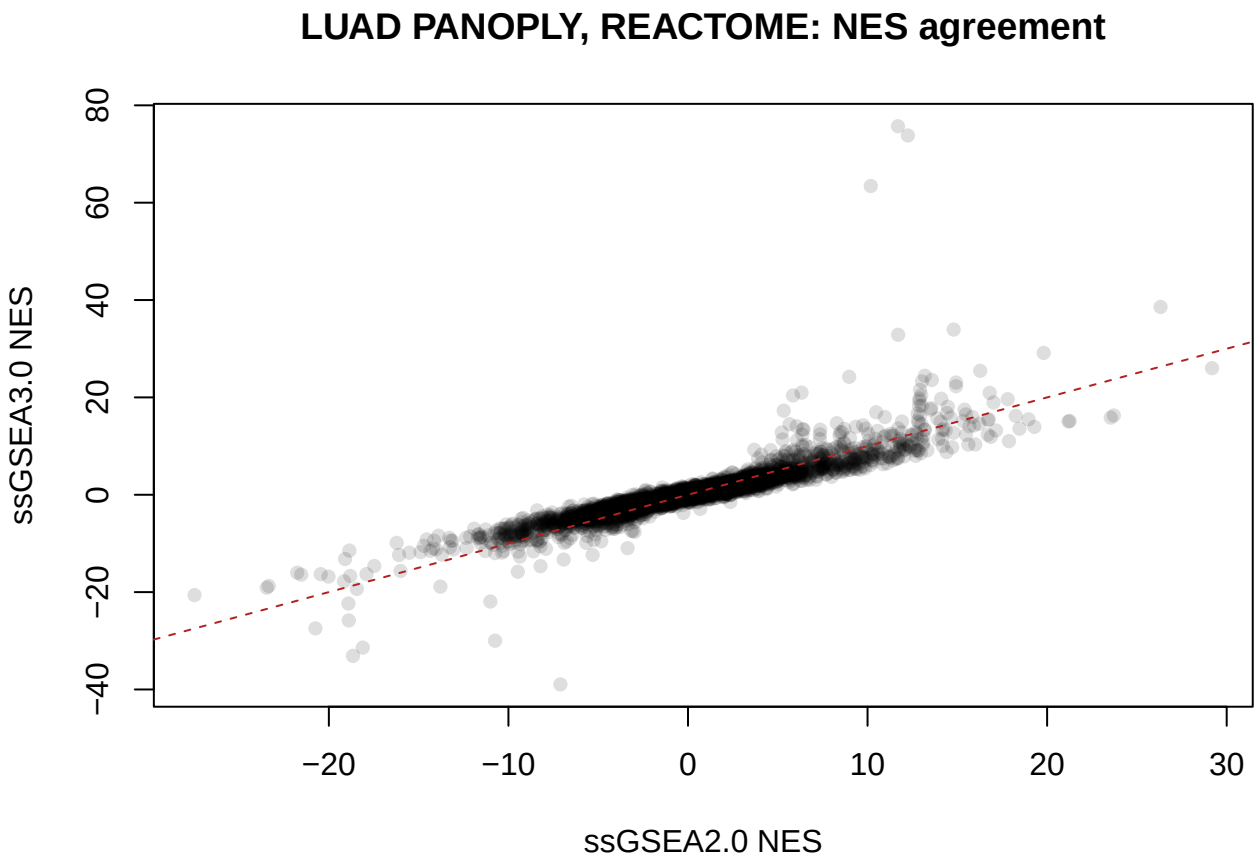
```
cmp.reactome <- readRDS(system.file(
  "extdata", "luad_reactome_ssgsea2_vs_ssgsea3_comparison.rds", package = "ssGSEA3"
))
nes2.reactome <- as.vector(cmp.reactome$ssgsea2.nes)
nes3.reactome <- as.vector(cmp.reactome$ssgsea3.nes)

cat("Gene sets compared:", length(cmp.reactome$gene.set.names), "\n")
#> Gene sets compared: 1167
cat("NES Pearson r:    ", round(cor(nes2.reactome, nes3.reactome), 3), "\n")
#> NES Pearson r:     0.887
cat("NES Spearman rho: ", round(cor(nes2.reactome, nes3.reactome, method = "spearman"),
 ↪  3), "\n")
```

```
#> NES Spearman rho:  0.966

sig2.reactome <- apply(cmp.reactome$ssgsea2.fdr < 0.05, 1, any)
sig3.reactome <- apply(cmp.reactome$ssgsea3.fdr < 0.05, 1, any)
cat("\nGene sets significant (FDR < 0.05 in >=1 sample), of",
 ↪  length(cmp.reactome$gene.set.names), "total:\n")
#>
#> Gene sets significant (FDR < 0.05 in >=1 sample), of 1167 total:
cat("  ssGSEA2.0:", sum(sig2.reactome), " ssGSEA3.0:", sum(sig3.reactome),
    " both:", sum(sig2.reactome & sig3.reactome),
    " ssGSEA2.0-only:", sum(sig2.reactome & !sig3.reactome),
    " ssGSEA3.0-only:", sum(!sig2.reactome & sig3.reactome), "\n")
#>   ssGSEA2.0: 1023  ssGSEA3.0: 805  both: 800  ssGSEA2.0-only: 223  ssGSEA3.0-only: 5
```

```
plot(nes2.reactome, nes3.reactome,
     xlab = "ssGSEA2.0 NES", ylab = "ssGSEA3.0 NES", pch = 16, col = "#00000022",
     main = "LUAD PANOPLY, REACTOME: NES agreement")
abline(0, 1, col = "firebrick", lty = 2)
```



The pattern is the same shape as HALLMARK's – `ssGSEA3.0` keeps the large majority of `ssGSEA2.0`'s significant calls (800 of 1023) while adding relatively few new ones of its own (5) – but at much larger absolute scale (1167 gene sets vs. HALLMARK's 50), exactly as the size/function-diversity hypothesis above predicts: a bigger, more heterogeneous database makes the same underlying calibration difference between the two engines far more visible.

Example 3: PTM-SEA on a PI3K-inhibitor perturbation dataset

The second example (bundled in the package's `example/` directory, alongside its source) is a site-centric phosphoproteomics dataset from the original PTM-SEA publication itself (Krug et al. 2019 – see the package `README` for the full citation): T47D breast cancer cells treated with the PI3Kα inhibitor BYL719 (vs. a DMSO control) for 6h and 24h (`PI3K_pert_logP_n2x23936.gct`), scored against PTMsigDB v2.0.0's flanking-sequence human signatures – a directional PTM-SEA analysis. The call is the same shape as the ssGSEA example above:

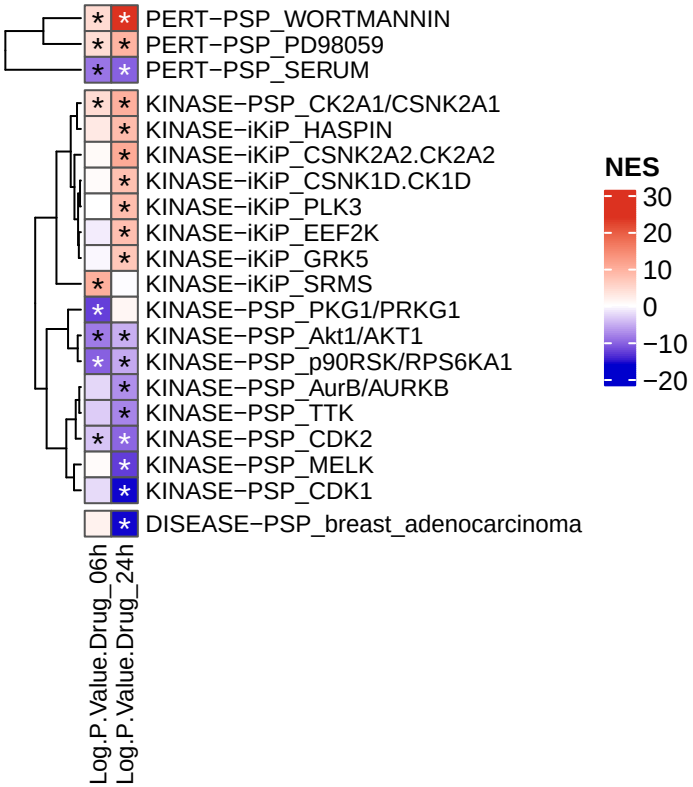
```
res.ptmsea <- ssGSEA3(
  input.ds = "example/PI3K_pert_logP_n2x23936.gct",
  output.prefix = "pi3k",
  gene.set.databases = "example/ptm.sig.db.all.flanking.human.v2.0.0.gmt",
  sample.norm.type = "rank", weight = 0, correl.type = "rank",
  statistic = "area.under.RES", min.overlap = 5, nperm = 1000, seed = 42
)
```

```
## as above, loading the precomputed (nperm=1000) output instead of
## re-running the ~9 min scoring call live
res.ptmsea <- readRDS(system.file("extdata", "pi3k_ssgsea3_result.rds", package =
 ↪  "ssGSEA3"))
res.ptmsea
#> <ssgsea_result> 569 gene sets x 2 samples
#>   score type: NES
```

PTMsigDB's `CATEGORY-name` convention (`KINASE-`, `PERT-`, `PATH-`, `DISEASE-`) drives its own row-grouping in `plot_ssgsea_heatmap()`, distinct from the HALLMARK-category grouping used above:

```
ht.ptmsea <- plot_ssgsea_heatmap(res.ptmsea, top_n = 10, fdr.max = 0.05,
                                 title = "PI3K perturbation -- PTMsigDB v2.0.0
                                  ↪  (ssGSEA3.0)")
ComplexHeatmap::draw(ht.ptmsea)
```

PI3K perturbation −− PTMsigDB v2.0.0 (ssGSEA3.0)



Comparison against the original ssGSEA2.0

Unlike the LUAD comparison above, `weight = 0` is used here deliberately, to isolate the bootstrap-vs-permutation effect on directional scoring from the separate weight-dependent issue that comparison demonstrated. The results below use the default `nperm = 1000` (run outside this vignette, since a full comparison run against the original, unmodified `ssGSEA2.0` code depends on software not part of this package):

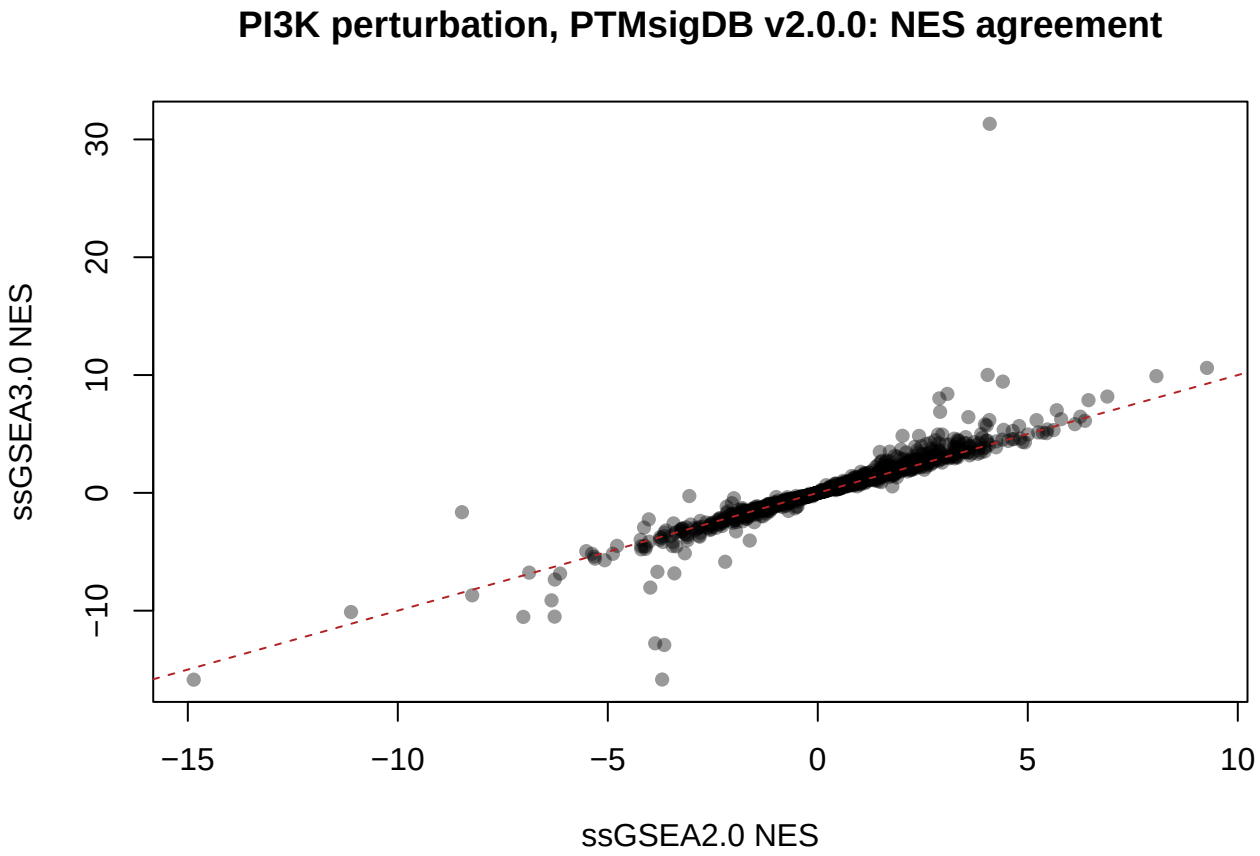
```
cmp <- readRDS(system.file(
  "extdata", "pi3k_ssgsea2_vs_ssgsea3_comparison.rds", package = "ssGSEA3"
))
nes2 <- as.vector(cmp$ssgsea2.nes)
nes3 <- as.vector(cmp$ssgsea3.nes)
fdr2 <- as.vector(cmp$ssgsea2.fdr)
fdr3 <- as.vector(cmp$ssgsea3.fdr)

cat("Gene sets compared:", length(cmp$gene.set.names), "\n")
#> Gene sets compared: 569
cat("NES Pearson r:    ", round(cor(nes2, nes3), 3), "\n")
#> NES Pearson r:     0.905
cat("NES Spearman rho: ", round(cor(nes2, nes3, method = "spearman"), 3), "\n")
#> NES Spearman rho:  0.993
cat("FDR Pearson r:    ", round(cor(fdr2, fdr3), 3), "\n")
#> FDR Pearson r:     0.966
cat("FDR Spearman rho: ", round(cor(fdr2, fdr3, method = "spearman"), 3), "\n")
#> FDR Spearman rho:  0.972

## a gene set counts as "significant" if FDR<0.05 in >=1 of the 2 samples --
```

```
## NOT the same as counting gene-set x sample cells, which would double-count
## any gene set significant in both samples
sig2 <- apply(cmp$ssgsea2.fdr < 0.05, 1, any)
sig3 <- apply(cmp$ssgsea3.fdr < 0.05, 1, any)
cat("\nGene sets significant (FDR < 0.05 in >=1 sample), of", length(cmp$gene.set.names),
 ↪  "total:\n")
#>
#> Gene sets significant (FDR < 0.05 in >=1 sample), of 569 total:
cat("  ssGSEA2.0:", sum(sig2), " ssGSEA3.0:", sum(sig3),
    " both:", sum(sig2 & sig3), " ssGSEA2.0-only:", sum(sig2 & !sig3),
    " ssGSEA3.0-only:", sum(!sig2 & sig3), "\n")
#>   ssGSEA2.0: 86  ssGSEA3.0: 127  both: 79  ssGSEA2.0-only: 7  ssGSEA3.0-only: 48
```

```
plot(nes2, nes3,
     xlab = "ssGSEA2.0 NES", ylab = "ssGSEA3.0 NES", pch = 16, col = "#00000066",
     main = "PI3K perturbation, PTMsigDB v2.0.0: NES agreement")
abline(0, 1, col = "firebrick", lty = 2)
```



`ssGSEA3.0` retains the large majority of `ssGSEA2.0`'s significant calls (79 of 86) while finding substantially more overall (127 vs. 86) – consistent with the confirmed power advantage for directional scoring described above, now demonstrated on a real, published dataset.

Are the additional calls biologically plausible?

A real power advantage should surface signatures that make sense for this specific perturbation, not an arbitrary scattering of unrelated biology. Of the 48 signatures significant under `ssGSEA3.0` but never significant (in either sample) under `ssGSEA2.0`, several sit directly on the PI3K/AKT/mTOR axis this dataset perturbs:

```
gained <- c(
  "KINASE-iKiP_PIK3R5", "KINASE-PSP_PDK1/PDPK1", "KINASE-PSP_SGK1",
  "KINASE-PSP_p70S6K/RPS6KB1", "PERT-PSP_TORIN1", "DISEASE-PSP_tuberous_sclerosis"
)
data.frame(
  signature = gained,
  NES_06h = round(cmp$ssgsea3.nes[gained, 1], 2),
  NES_24h = round(cmp$ssgsea3.nes[gained, 2], 2),
  FDR_06h = signif(cmp$ssgsea3.fdr[gained, 1], 2),
  FDR_24h = signif(cmp$ssgsea3.fdr[gained, 2], 2),
  row.names = NULL
)
#>                       signature NES_06h NES_24h FDR_06h FDR_24h
#> 1             KINASE-iKiP_PIK3R5   -0.78    3.87  0.8200 2.2e-02
#> 2          KINASE-PSP_PDK1/PDPK1   -0.19    4.84  0.9600 1.1e-03
#> 3                KINASE-PSP_SGK1   -4.52   -0.97  0.0092 6.9e-01
#> 4      KINASE-PSP_p70S6K/RPS6KB1   -3.25   -3.55  0.0910 4.3e-02
#> 5                PERT-PSP_TORIN1    4.11    3.63  0.0440 3.4e-02
#> 6 DISEASE-PSP_tuberous_sclerosis   -2.00   -6.84  0.3700 3.6e-06
```

PIK3R5 is a PI3K regulatory subunit itself; PDK1 and SGK1 are AKT-family kinases immediately downstream of PI3K; p70S6K is the canonical mTORC1 output kinase; TORIN1 is a PTMsigDB perturbation signature for an mTOR inhibitor; tuberous_sclerosis is a disease signature driven by TSC1/2, a direct mTORC1 regulator. This lines up with the original paper's own reported result for this exact experiment: PTM-SEA "detected signatures of compounds inhibiting PI3K as well as targets downstream of PI3K (AKT, MAPK/ERK) covering a substantial fraction of the PI3K pathway" (Krug et al. 2019). p70S6K and SGK1's substrate signatures moving negative is consistent with declining mTORC1/AKT-family output as BYL719 inhibition takes hold; PDK1 and PIK3R5 trending positive by 24h instead is consistent with the compensatory feedback reactivation of upstream RTK/PI3K-node signaling well documented as a response to PI3K inhibitors specifically (rather than a random, unrelated direction) – so even the "wrong-direction" hits are still on-pathway. The additional calls cluster around the perturbed pathway rather than appearing as unrelated noise, which is exactly what ssGSEA2.0 under-detected here.

A related, separate observation: PERT-PSP_WORTMANNIN – a PTMsigDB signature for an unrelated experiment's treatment with wortmannin, another PI3K inhibitor – was already significant under ssGSEA2.0 at 24h (FDR 0.014), so it is not part of the "newly significant" tally above. But ssGSEA3.0 closes a near-miss at 6h (FDR 0.042 under ssGSEA2.0, 0.026 under ssGSEA3.0) and dramatically strengthens the 24h call (NES 4.1 to 31.3) – close to a positive control for the same PI3K/mTOR-inhibitor resemblance the newly-significant calls above point to.

Session info

```
sessionInfo()
#> R version 4.4.2 (2024-10-31)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS Sequoia 15.7.7
#>
#> Matrix products: default
#> BLAS:
 ↪  /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK:
 ↪  /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
 ↪  LAPACK version 3.12.0
```

```
#>
#> locale:
#> [1] C
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] ssGSEA3_1.0.0  testthat_3.3.2
#>
#> loaded via a namespace (and not attached):
#>   [1] remotes_2.5.0               rlang_1.1.7
#>   [3] magrittr_2.0.4              clue_0.3-66
#>   [5] GetoptLong_1.0.5            otel_0.2.0
#>   [7] verification_1.45           matrixStats_1.4.1
#>   [9] compiler_4.4.2              png_0.1-8
#>  [11] vctrs_0.6.5                 maps_3.4.3
#>  [13] profvis_0.4.0               pkgconfig_2.0.3
#>  [15] shape_1.4.6.1               crayon_1.5.3
#>  [17] fastmap_1.2.0               magick_2.9.1
#>  [19] XVector_0.46.0              ellipsis_0.3.2
#>  [21] promises_1.5.0              rmarkdown_2.30
#>  [23] sessioninfo_1.2.2           UCSC.utils_1.2.0
#>  [25] tinytex_0.58                purrr_1.2.1
#>  [27] xfun_0.55                   zlibbioc_1.52.0
#>  [29] cachem_1.1.0                GenomeInfoDb_1.42.1
#>  [31] jsonlite_2.0.0              later_1.4.5
#>  [33] DelayedArray_0.32.0         parallel_4.4.2
#>  [35] cluster_2.1.8               R6_2.6.1
#>  [37] RColorBrewer_1.1-3          boot_1.3-31
#>  [39] pkgload_1.4.0               brio_1.1.5
#>  [41] GenomicRanges_1.58.0        knitr_1.51
#>  [43] Rcpp_1.1.1                  SummarizedExperiment_1.36.0
#>  [45] iterators_1.0.14            fields_17.1
#>  [47] usethis_3.1.0               IRanges_2.40.1
#>  [49] flowCore_2.18.0             httpuv_1.6.16
#>  [51] Matrix_1.7-4                tidyselect_1.2.1
#>  [53] yaml_2.3.12                 rstudioapi_0.17.1
#>  [55] abind_1.4-8                 doParallel_1.0.17
#>  [57] codetools_0.2-20            dtw_1.23-1
#>  [59] miniUI_0.1.2                pkgbuild_1.4.5
#>  [61] lattice_0.22-6              tibble_3.3.1
#>  [63] Biobase_2.66.0              shiny_1.12.1
#>  [65] withr_3.0.2                 evaluate_1.0.5
#>  [67] desc_1.4.3                  proxy_0.4-29
#>  [69] urlchecker_1.0.1            circlize_0.4.16
#>  [71] pillar_1.11.1               MatrixGenerics_1.18.0
#>  [73] foreach_1.5.2               stats4_4.4.2
#>  [75] generics_0.1.4              rprojroot_2.1.1
#>  [77] S4Vectors_0.44.0            gtools_3.9.5
```

```
#>  [79] xtable_1.8-4                glue_1.8.0
#>  [81] tools_4.4.2                 fs_1.6.6
#>  [83] dotCall64_1.2               grid_4.4.2
#>  [85] CircStats_0.2-7             RProtoBufLib_2.18.0
#>  [87] devtools_2.4.5              colorspace_2.1-2
#>  [89] GenomeInfoDbData_1.2.13     cmapR_1.18.0
#>  [91] cli_3.6.5                   spam_2.11-3
#>  [93] cytolib_2.18.2              S4Arrays_1.6.0
#>  [95] viridisLite_0.4.2           ComplexHeatmap_2.22.0
#>  [97] dplyr_1.1.4                 digest_0.6.39
#>  [99] BiocGenerics_0.52.0         SparseArray_1.6.0
#> [101] rjson_0.2.23                htmlwidgets_1.6.4
#> [103] memoise_2.0.1               htmltools_0.5.9
#> [105] lifecycle_1.0.5             httr_1.4.7
#> [107] GlobalOptions_0.1.2         mime_0.13
#> [109] MASS_7.3-64
```